

May 2026

Surprising Effectiveness of Self-Demonstrations in Enhancing Schema-Ontology Mapping with LLMs

Siddhesh THOMBRE^a Manasi PATWARDHAN^b and Sunita SARAWAGI^b^a*TCS Research*^b*IIT Bombay*

Abstract. Integrating heterogeneous relational databases into a centralized ontology remains a persistent challenge in enterprise knowledge representation, primarily due to semantic heterogeneity, cryptic schema naming, missing metadata, and the abstraction gap between relational schemas and ontological models. Although large language models (LLMs) offer strong semantic reasoning capabilities, we show that directly applying them through one-shot prompting or naïve multi-stage pipelines leads to poor performance for schema-ontology mapping. This paper presents a self-demonstration-driven approach that combines a neuro-symbolic task decomposition with a novel mechanism for automatically generating pattern-guided, dependency-aware demonstrations to address this integration challenge. Our approach incorporates two key strategies to achieve substantial accuracy gains over existing LLM-based schema integration methods: (i) a neuro-symbolic decomposition of the task into cascaded sub-tasks, where symbolic constraints structure the search space and LLMs perform semantic reasoning within each focused sub-task, and (ii) self-generated demonstrations guided by domain-agnostic patterns to supervise each sub-task. Experiments on three of the most challenging scenarios from the RODI benchmark show that our approach achieves state-of-the-art performance, substantially outperforming (~25 percentage points F1 improvements) both traditional schema-to-ontology mapping techniques and recent LLM-based schema-to-ontology and schema matching approaches. Ablation studies further reveal the significant benefits of pattern-guided self-demonstrations and the complementary benefits of neuro-symbolic task decomposition.

Keywords. Database-Ontology mapping, data integration, enterprise search, R2RML generation

1. Introduction

Enterprises increasingly seek to represent organizational knowledge through a single unified ontology so that analytics, search, and reasoning can operate over a semantically consistent enterprise view [1,2,3]. In practice, however, enterprise data is distributed across multiple heterogeneous and independently evolving relational databases. As a result, querying across these sources requires mapping each database schema to a centralized ontology (Example illustrated in Figure 1). Doing this manually is a time-consuming, error-prone, and cognitively demanding task at enterprise scale, given the large number of ontology entities, database tables, and columns involved.

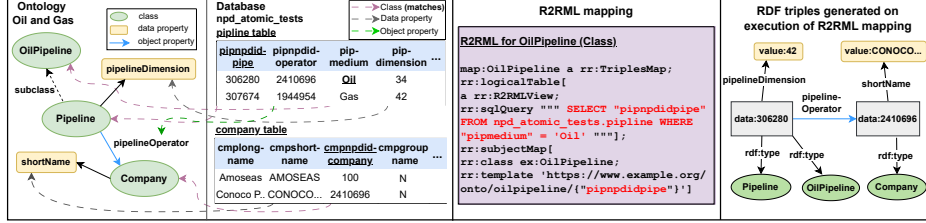


Figure 1. RDB-to-Ontology Mapping Example

Traditional automated mapping systems [4,5] rely heavily on lexical similarity and hand-crafted templates; consequently, they often miss deeper semantic correspondences, struggle with cryptic or noisy schema elements, and fail when key metadata is incomplete. Large language models (LLMs) are a natural fit for this task because they offer broad semantic reasoning capabilities [6,7,8,9] that can bridge representational gaps between schemas and ontologies. Yet our early experiments show that directly prompting an LLM to produce complete DB-to-ontology mappings in one shot yields poor results. When the overall task is organized in a neuro-symbolic manner, where symbolic structure decomposes the task into stages and constrains the search space while LLMs perform semantic reasoning within each stage, the model makes better local decisions and produces better mappings. However, the benefits yielded by this staged LLM driven approach in zero-shot setting leads to sub-optimal results as inaccuracies at each LLM stage accumulate, leading to a decline in overall performance. To improve accuracy, we sought to harness the in-context learning (ICL) [10] capability of LLMs to adapt to new tasks or domains using demonstrations with a few input-output exemplars. We observe that manually curated demonstrations substantially improved accuracy, but they are labor-intensive and not scalable due to the need for domain and task expertise.

Matchmaker [11,12], an approach for schema matching, demonstrates the capability of LLMs to automatically generate few-shots for self-improvement. However, extending their approach for our pipeline does not yield us performance on par with the manually curated few-shots. This was mainly because matchmaker’s approach tend to generate similar-looking demonstrations, with limited semantic variations in the matches.

This leads to key insight of this work: We allow LLM to generate self-demonstrations while constraining generation using typical reusable, domain-agnostic patterns observed across real data integration cases. We define pattern families that capture common regularities and guide the LLM to instantiate them, for each stage, to generate self-demonstrations. For example, a class in an ontology matches a DB table following either an exact string-based, syntactic, semantic, domain-based, or categorical-column-value-pair-based match. An object property connecting two classes, in contrast, matches with either the foreign key column of the matched table of the domain class or the columns of the junction table that links the tables matched with the domain and range classes. The demonstrations are created in a dependency-aware manner: outputs from earlier stages (such as class/table alignments) are carried forward while generating demonstrations for downstream sub-tasks (e.g., property matching and SQL-view mapping). We show that our staged process of automatic generation and utilization of pattern-grounded, dependency-aware self-demonstrations, which are diverse and less noisy, is effective and is the primary driver of performance gains for LLM driven database to ontology map-

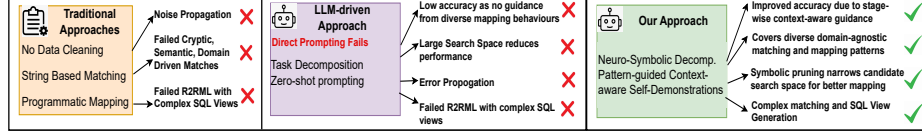


Figure 2. Motivation behind Our Approach of DB-to-Ontology Mapping

ping task. Although our approach invokes LLMs at multiple stages, database to ontology mapping is typically a one-time integration process whose resulting mappings are reused extensively across downstream queries and applications. As a result, we prioritize accuracy and semantic correctness over execution efficiency or compute cost in enterprise settings. The key contributions of our approach are summarized as follows:

- We propose a neuro-symbolic decomposition based approach that combines the strengths of traditional symbolic approaches such as Milan [13] with the power of modern LLMs. The database-to-ontology mapping task is systematically decomposed into multiple stages and fine-grained sub-stages, each executed using self-demonstration-guided LLMs.
- We introduce a pattern-guided context-aware self demonstration method that automatically constructs diverse, stage-specific in-context exemplars, with one-time domain agnostic pattern curation.
- Our approach achieves state-of-the-art results on the three most challenging scenarios from the RODI benchmark [4] (geographical, conference and oil and gas), demonstrating substantial improvements over direct prompting and prior automated baselines (~ 25 percentage points in F1 score) and even slightly better results than manually curated few-shots.
- We show via detailed ablations that our approach benefit from neuro-symbolic task decomposition and self demonstrations.

2. Related Work

DB-Ontology Mapping. Early systems such as BootOX [14], Ontop [15], MIRROR [16], and D2RQ [17] derive a bootstrapped ontology from a database and then try to map this ontology with the schema, which is not a realistic setting. Subsequent tools like COMA++ [18], IncMap [19], A4MO [5], and MILAN [13] rely mainly on lexical/structural similarity with heuristic pruning and template-based mapping. These approaches degrade under cryptic nomenclature, incomplete metadata (e.g., missing FKs), and when semantic correspondences transcend string similarity. Recent LLM-based methods (Table-to-KG [20] and LLM4VKG [21]) introduce learned semantic matching with LLMs but often stop at alignment, keep mapping template-driven, or do not provide targeted, stage-specific guidance to the LLM. In contrast, our approach combines symbolic decomposition and pruning with LLM-based semantic reasoning, and further provides targeted, stage-specific guidance through pattern-guided self-demonstrations.

Schema Matching. In the DB-to-ontology mapping task, the initial correspondence identification step closely resembles schema matching. LLM-based schema matching commonly follows “retrieve candidates then re-rank with an LLM,” as in Re-

Match [22] and Magneto [23]. Matchmaker [11,12] further explores automated few-shot self-improvement. Yet schema matching is typically simpler than DB–ontology mapping given the abstraction and granularity mismatch between schemas and ontologies. Matchmaker’s few-shot generation approach when adapted to our setting leads to sub-optimal results. In our setting, automated few-shots help only when they are diverse, scenario-specific, and dependency-aware, characteristics we enforce via self-demonstrations.

Takeaway. Prior literature indicates that LLMs can capture semantics beyond strings, but it underplays how to combine semantic reasoning with symbolic decomposition and how to feed LLMs the right in-context signals. We operationalize this through a neuro-symbolic design with pattern-guided self-demonstrations, automatically synthesized, diverse, and scenario-specific exemplars that steer the model across mapping stages.

3. Background and Problem Definition

We formalize the problem and recall essential background on ontology structure and schema-to-ontology mappings. We are given as input: (i) an Ontology $\mathcal{O}$ consisting of a set-of classes C such as Pipeline in Figure 1, subclasses $C_S \rightarrow C$ (e.g. OilPipeline) for a subset of classes C , data properties D_C (e.g. pipelineDimension) with domain as a class (e.g. Pipeline) and range as its data type (such as ‘float’), and object properties O_{C_D, C_R} (e.g. pipelineOperator) linking two classes, a domain class C_D (e.g. Pipeline) and a range class C_R (e.g. Company), and (ii) Relational Database schema $\mathcal{D}$ consisting of entities such as tables T (e.g. pipeline), columns belonging to those tables C_T (e.g. pipemedium), and data values of those columns V_{C_T} (e.g. ‘Oil’).

An Ontology-DB mapping entails associating ontology concepts (class, data properties and object properties) with corresponding database entities (tables, columns). Typical regularities include: classes map to table primary keys or filtered table subsets via categorical columns; subclasses map to column–value filters in or adjacent to superclass tables; data properties map to columns in class tables or one-hop neighbors; object properties maps to foreign keys or junction-table keys connecting domain and range.

A snippet of such a mapping task between a subset of oil and gas ontology made for the Norwegian Petroleum Directorate (NPD) and a relational DB is shown in Figure 1. Mappings are expressed in R2RML (Relational to Resource Description Framework Mapping Language) [24]. Our goal is to generate a set of R2RML mappings consisting of SQL views, with subject and object columns as illustrated in Figure 1 such that execution of the R2RML mapping leads to RDF triples, populating $\mathcal{O}$ with the data in $\mathcal{D}$.

4. Self-Demonstration Driven Approach

4.1. Overview of Neuro-Symbolic Decomposition

Motivated by the observation that despite long context support, current LLMs struggle to produce accurate R2RML mappings in a single, one-shot pass, we decompose the DB-to-Ontology mapping task into two stages: (i) Matching: Identifying the most relevant target entities in the DB schema $\mathcal{D}$ that correspond to a source concept in the ontology $\mathcal{O}$, (ii) Mapping: Constructing R2RML mappings for each ontology concept $\mathcal{O}$ by gener-

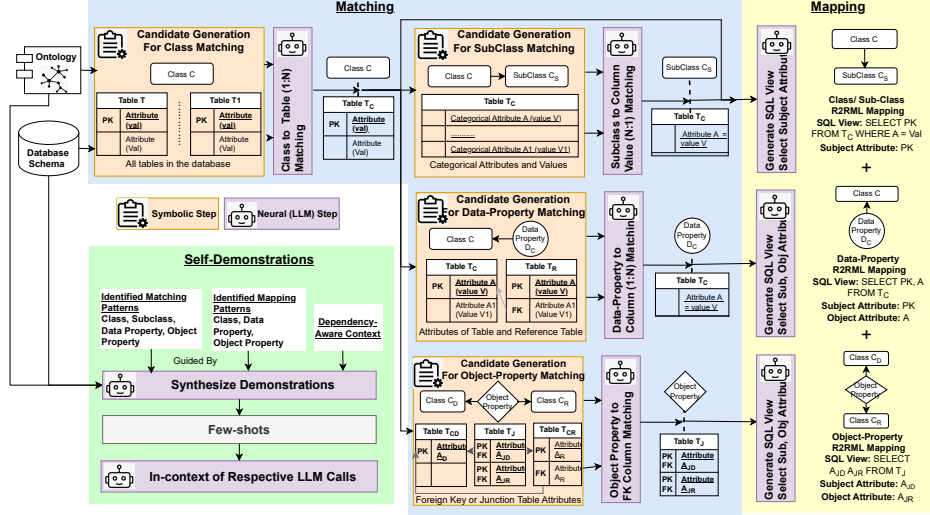


Figure 3. Overview of Self-Demonstration Driven Mapping Approach. Decomposition: (i) **Matching**, (ii) **Mapping** and **Self-Demonstrations**. T_C : Table matched with class C , C_S : Subclass of Class C , D_C : Data-property of Class C , T_K : Table referring to T_C , T_J : Junction Table

ating SQL views over matched schema entities. We infer missing foreign keys in $\mathcal{D}$ and domain or range declarations in $\mathcal{O}$ when such metadata is absent, before serving them as an input to the above stages. Both stages are further divided into sub-stages for aligning ontology concepts, preserving their dependencies. Similar to Milan [13], we address the vast schema–ontology search space by performing matching in a top-down, stage-wise manner, progressively pruning candidates using symbolic constraints. We decompose the overall task into sub-stages based on ontology entity types and construct focused candidate sets using rule-based reasoning, substantially reducing the number of database entities taken into consideration as candidates compared to full database schema. Unlike Milan, we tightly integrate this symbolic decomposition with neural component by invoking LLMs at each sub-stage, equipped with carefully scoped contextual information. Operating over these focused candidate sets allows the LLM to reason within a constrained and relevant context, leading to more accurate and robust mappings [25,26,27,28].

(i) *Matching*. Matching proceeds in four sub-stages in the order: class, subclass, data property, and object property. Class matching is a two-step process that first selects the best matching table, then additional matching tables if present, are identified. Subclass matching is invoked only when the superclass is matched and searches categorical column–value pairs in the superclass tables and in one hop related tables. Data property matching draws candidates from tables matched to the domain class and from one hop related tables. Object property matching runs only when both domain and range classes are matched, considering candidates from the domain and range tables, one hop related tables, and keys of the junction table, when applicable. For example, in the NPD database–ontology pair from the RODI benchmark (see Section 5.1), symbolic pruning reduces the candidate search space by approximately 95% for data property matching and 85% for object property matching stages, thereby enabling more focused and accurate matching.

(ii) *Mapping*. Mapping is decomposed into class, data property, and object property mapping. For each ontology concept, the model generates a SQL view and identifies

the subject and object columns required by R2RML using the matches obtained in the matching stage for that concept. Class mapping selects subject identifiers from primary keys of the matched class tables. For subclasses, the mapping involves generating SQL filter conditions (e.g., category predicates) that distinguish subclasses when they match to a shared table. Data property mapping combines the matched value column with the identifiers of the domain class, while object property mapping links identifiers of the domain and range classes through the matched relation columns, including junction table paths when required. Per concept outputs are aggregated into the final R2RML mapping file. The matching and mapping prompts, along with the symbolic steps, are provided in the repository¹. Figure 3 provides an overview of this decomposition.

This sequential stage-wise structure improves mapping quality but the decisive gains arise when each stage is augmented with self-generated, pattern-guided demonstrations. These demonstrations are instantiated on the input DB–ontology pair and supplied in-context at the point of use, improving decision fidelity across the above stages.

4.2. Why Pattern-guided Context-aware Self-demonstrations.

Our initial zero-shot LLM experiments for the matching and mapping sub-tasks yield sub-optimal results. We observe that manually curated exemplars for In-Context Learning (ICL) lead to substantial performance improvements. However, this manual curation is labor-intensive and not scalable for distinct DB–Ontology pairs since it requires domain and task expertise. Hence, we seek methods of auto-generation of few-shots.

One of the Schema Matching approaches, Matchmaker [11,12], utilized automatic few-shot generation for self-improvement in schema-to-schema matching. We observe that adapting Matchmaker for schema-to-ontology mapping, often produce exemplars concentrated around a narrow style of semantic similarity, with limited coverage of the broader matching and mapping behaviors required for DB-to-ontology integration (Details in Section 5.3). Database to ontology mapping problem is fundamentally harder than schema matching because databases and ontologies represent knowledge at different levels of abstraction and granularity. Relational schemas are designed for efficient storage and query execution, and therefore expose low-level, fine-grained structures such as tables and columns. Ontologies, in contrast, encode high-level domain concepts, constraints, and relationships. This mismatch is amplified in enterprise settings where naming conventions vary across systems, semantically related entities use dissimilar labels, and similarly named entities may carry different meanings depending on context. This leads to distinct scenarios or patterns of this mapping task, with much higher variations than schema matching. Thus, diversity in few-shots is critical to cover varied matching and mapping scenarios [29,30]. Further, each sub-stage of schema-to-ontology mapping task relies on information produced by previous ones and thus, synthesizing demonstrations for any sub-stage must incorporate dependency-aware context (i.e., the outputs from earlier stages) to ensure coherence and correctness of the demonstrations. Thus, unlike schema-matching pipelines, the schema-to-ontology mapping stages exhibit strong sequential dependencies to avoid error-propagation through stages.

To avoid manual intervention but at the same time retain the quality and diversity of few-shots, we devise the self-demonstrations method to auto-generate few-shots for

¹<https://anonymous.4open.science/r/Schema-Ontology-Mapper-4BC1/>

any given DB-ontology pair with minimal one-time guidance using a reusable library of domain-agnostic patterns and dependency-aware context.

4.3. Synthesis of Self-Demonstrations

We provide the LLM with an exhaustive set of predefined domain-agnostic input patterns that demonstrate distinct strategies of matching and mapping (patterns are provided in the repository). With these patterns in context, we task the LLM to instantiate them on the given DB-ontology pair. Thus, starting from generic patterns, we generate a comprehensive and diverse set of few-shots for any given DB-ontology pair. With this approach, we ensure that the generated few-shots cover a wide array of matching and mapping cases.

A pattern is a template that captures a matching/mapping behaviour. The pattern library encodes recurring correspondence behaviors seen across DB-ontology integration tasks: lexical and semantic alignment patterns (e.g., exact, paraphrased, or abbreviation-based matches), structural patterns (e.g., object properties realized through foreign keys or junction tables), and value-constrained patterns (e.g., subclasses represented as filtered subsets based on categorical column values). We also include mapping-side patterns for SQL view construction, such as projection-only mappings, join-based mappings, and mappings requiring value normalization or conditional filtering. This pattern-oriented organization makes each demonstration explanatory rather than merely illustrative: the few-shot not only states the final match or SQL fragment, but also reflects the underlying pattern that justifies it. Patterns also act as a controllable diversity mechanism. During demonstration synthesis, we explicitly prompt the LLM to cover different pattern families instead of selecting random examples. As a result, the final in-context set spans both straightforward and complex behaviors, improving robustness when the target concept exhibits ambiguous naming, implicit relations, or non-trivial SQL transformations.

Few-shots are generated for each matching and mapping sub-stage sequentially by providing stage specific patterns (prompt is provided in repository). For each sub-stage, LLM leverages the outputs of the relevant prior stage(s) to further improve the quality of the demonstrations. For example, for generation of few-shots for subclass, data and object property matching step, class matching output serves as additional input. Whereas for data and object property mapping, the output of data and object property matching serve as an input (details are provided in Figure 4).

This dependency-aware context improves demonstration quality for two reasons. First, it guides the LLM to focus on candidates that are structurally consistent with earlier decisions (e.g., data-property examples are grounded in tables already selected for the domain class), which lowers semantically plausible but invalid matches. For example, consider the data property `hydrocarbonType`, whose domain is the class `Discovery` and whose range is string. From the earlier class matching stage, the class `Discovery` has

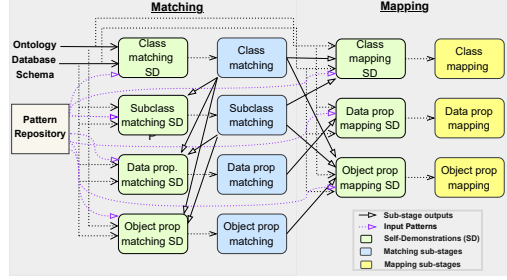


Figure 4.: Dependency-Aware context for Self-Demonstrations

been aligned with the `discovery` table. During the self demonstration generation step for property matching, this prior alignment is included as contextual information. By making the class to table correspondence explicit, the context directs the LLM’s attention toward columns associated with the `discovery` table, leading it to focus on candidates such as `discovery.dshctype` rather than exploring other semantically plausible but structurally less aligned columns such as `dscarea.dshctype` and `fldarea.dshctype`.

Second, because established upstream decisions (e.g., class-table alignments) are passed into downstream demonstration construction, demonstrations across downstream tasks (e.g., data property alignment / mapping) become more consistent and correct. For example, consider the data property `wellboreStratumBottomDepth`, whose domain is the class `WellboreStratum` and whose range is decimal. In an earlier class-matching stage, the `WellboreStratum` class is aligned with the `wellbore_formation_top`, `strat_litho_wellbore` tables using richer contextual signals, giving these class table alignments a higher likelihood of being correct. When these higher confidence upstream decisions are incorporated as contextual input during the self demonstration generation step for property matching, they provide reliable structural anchors for downstream reasoning. As a result, the LLM constructs property-level demonstrations under the assumption that `wellboreStratumBottomDepth` should be interpreted within the context of the `wellbore_formation_top` and `strat_litho_wellbore` tables, leading to consistent demonstrations that align the property with `wellbore_formation_top.lsubottomdepth` and `strat_litho_wellbore.lsubottomdepth`. In contrast, without incorporating these upstream alignments, the LLM may either arbitrarily select a single table or treat multiple semantically plausible columns as equally valid, increasing the likelihood of generating inconsistent or incorrect property-level demonstrations. Thus, dependency-aware few-shot generation improves example quality and consistency while limiting error propagation across sub-stages. Figure 4 summarizes the context used for self-demonstration synthesis across stages.

In manual few-shot curation, an expert selects demonstrations, each corresponding to a distinct matching or mapping pattern in the DB-ontology pair. Identifying representative examples for new pairs is challenging and time-consuming. In contrast, our automatic approach instantiates at most one example per applicable pattern. Both settings use the same number of few-shot examples (one per pattern). We use the same fixed domain-agnostic pattern set across all input DB-ontology pairs; new domains require no expert-authored templates, though experts may optionally add rare missing patterns.

5. Experimentation

5.1. RODI Benchmark

We use RODI (Relational-to-Ontology Data Integration) [4], benchmark to test the R2RML generation task. This benchmark has 18 test scenarios belonging to three domains. Each scenario consists of a DB, an ontology and set of evaluation queries to test expected results.

Name	Classes	Props.	Tables	Cols.	FKs	Queries
Conference	23	77	66	125	–	39
Mondial	49	71	42	160	60	50
Npd	300	350	70	1000	100	439

Table 1.: RODI Scenarios

We experiment with one representative scenario from each domain, viz., Mondial for Geographical Data, NPD-Atomic for oil and gas, and Conference-no-FK, the most challenging scenario within each domain [13]. Table 1 presents the statistics for the DB entities and ontology concepts of these scenarios. These span large DB schema and ontology of significant complexity, providing a realistic simulation of enterprise-level integration challenges. The Conference-no-FK scenario is especially difficult because DB schema lacks foreign key definitions, thus introducing ambiguity and noise into the task. For example, `was_a_program_committee_of` column of the `Committee` table should ideally be a foreign key of the `Conference_volume` table, but this relationship is missing in RODI. The mapping of data and object properties of the Mondial scenario exhibits a high degree of complexity, involving domain and range definitions, with logical combinations of multiple classes. For example, the domain class of `type data property` is defined as a complex expression involving unions, intersections and negations of classes, such as `"((GeographicalThing or Membership) and (not (City)) and (not (Continent)) and (not (Estuary)) and (not (River)) and (not (Source)))"`. It also involves object properties sharing identical domain and range classes. For example, the object properties `neighbor` and `dependentOf` have the same domain and range class as `Country`. The NPD-Atomic scenario represents the most complex case with very large ontology and DB schema. This scenario is characterized by a large number of one-to-many (1:n) mapping and individual ontology concepts must often be mapped to a union of several DB entities, to retrieve complete and correct results. For example, the mapping of `productionYear` data property is defined as a union over columns of 5 different tables.

Metric: We adopt the evaluation framework of the RODI benchmark. For each scenario a series of query pairs test a range of mapping challenges which are the result of distinct naming conventions of DB and ontology, structural conflicts caused by process of DB normalization, varied implementations of class hierarchies in an ontology, or semantic heterogeneity stemming from the conflict between a database’s closed-world and an ontology’s open-world assumption (object-relational gap). Every query pair consists of (i) a SPARQL query (test query) that runs against the RDF ontology data, resulting after execution of the generated R2RML, and (ii) a semantically equivalent SQL query (reference query) evaluated against the corresponding DB. For each query pair, the local per-query F-measure is computed by comparing reference tuples (SQL execution results) with result tuples (SPARQL execution results). The overall score reported for each scenario is the average of F-measures across test queries. More details are provided in [4]. We additionally report an average score (Avg), defined as the mean of F-measures computed over all queries across the selected test scenarios.

5.2. Baselines

Following approaches serve as our baselines: (i) traditional top-performing DB-to-Ontology mapping (Milan [13] and A4MO [5]) (ii) LLM-based DB-to-Ontology mapping (LLM4VKG [21] and table-to-KG [20]) and (iii) LLM-based state-of-the-art schema matching (Magneto [23]). **AM4O:** Automap [5] converts the DB schema into a putative ontology, where tables become classes, columns become data properties, and foreign keys become object properties. The classes and data properties of the putative and target ontology are matched using string similarity metrics. The alignment extension step refines these initial matches by analyzing the structural relationships between

the DB and ontology concepts. Finally, SQL queries for logical tables are programmatically derived using the DB paths identified during the alignment extension step. Automap’s alignment extension requires defined foreign keys, rendering it ineffective when absent. String similarity struggles with cryptic names or complex domains, and initial matching errors can propagate to subsequent alignment extensions. We use the results provided in [4]. **MILAN**: MILAN [13] uses Levenstein distance² to match DB-ontology correspondences and to identify optimal matches using combinatorial optimization. MILAN also generates the R2RML mappings programmatically using a set of predefined templates for the SQL views. It is currently the state-of-the-art traditional approach for the DB-to-ontology mapping task. We use the results provided in [4]. **Table-To-KG**: Table-to-KG [20] uses LLMs to match table columns with semantic concepts in a knowledge graph. For each DB column, the top-k (k=50) ontology concepts are ranked with the highest cosine similarity of text embeddings. We generate the embeddings with Gemini-embedding-001³. The top-k matches are re-ranked with an LLM and a sliding window approach (step size = 10, window size = 20). Chain-of-Thought with self-consistency [31] (n = 3 samples) is used to retrieve multiple rankings, which are subsequently fused by reciprocal rank fusion to select the top-ranked match. Although the original work uses Llama-3-70B⁴ for re-ranking and GPT-4o for completion, for a fair comparison, we use Gemini-2.0-flash for all stages. Also, this method only provides matching, to obtain mapping, we use the programmatic mapping approach like Milan. **LLM4VKG**: LLM4VKG [21] is a recent LLM-based approach to construct Virtual Knowledge Graphs (VKGs), consisting of DB, ontology and mappings. It performs the alignment with three modules. Retriever finds semantically similar ontology candidates using a sentence similarity model. Matcher refines matches and assigns a matching degree (High, Medium, Low) via an LLM. Finally, Namer generates new ontology terms when no suitable match exists. As a baseline, we use the results provided in [21]. **Magneto**: Magneto [23] is the state-of-the-art approach that leverages LLMs for the schema matching task, with the assumption of having a single table in the source and the target schema. To use this approach as one of the baselines, we customize it for matching ontology concepts to DB entities and execute mapping with Milan’s programmatic approach. For each concept in the ontology, we rank all entities in the DB using a score provided by a retriever (MPNet [32]), performing a semantic match. Then, for each concept in the ontology, the top-k ranked DB entities are re-ranked by prompting the LLM. We customize their prompt, originally designed for schema matching to schema-ontology matching. They use GPT-4o-mini⁵ for LLM re-ranking. However, for a fair comparison, we keep the LLMs consistent with those we use for our approach. **Matchmaker**: Matchmaker [11,12] is an LLM-based approach for schema matching. It involves a multi-step pipeline comprising candidate generation (using semantic retrieval and LLM reasoning), LLM-based candidate refinement, and confidence scoring. A key feature of Matchmaker is LLM self-improvement through automated in-context example selection. It identifies a set of ‘easy’ (having a semantic similarity score > 0.95 with target schema entities) and ‘challenging’ (exhibiting the lowest semantic matches) source schema entities. The complete matching pipeline is then executed for each identified entity, with all

²https://en.wikipedia.org/wiki/Levenshtein_distance

³<https://developers.googleblog.com/en/gemini-embedding-available-gemini-api/>

⁴<https://huggingface.co/meta-llama/Meta-Llama-3-70B>

⁵<https://platform.openai.com/docs/models/gpt-4o-mini>

intermediate outputs stored. An LLM evaluator assesses the final target schema entity match, assigning a relevance score from 0 to 5. The intermediate traces from the top-n source entities with the highest evaluation scores are selected as in-context examples. To assess the effectiveness of our self-demonstration generation strategy, we compare it with Matchmaker’s few-shot example selection strategy adopted and integrated in our pipeline. Specifically, for each stage in our approach, we obtain DB schema matches under zero-shot conditions, for ontology concepts selected using Matchmaker’s method. These results are then evaluated by an LLM evaluator, and the top-n scoring ontology concepts and their matches are chosen as in-context examples for each stage.

5.3. Results and Discussion

RQ1: How do LLM-driven approaches compare with traditional approaches? Direct single-pass LLM prompting is insufficient for obtaining complete DB-to-ontology mapping, as evidenced by the -*Matching+Mapping* ablation in Table 2, which achieves only 0.10 average F1 score and underscores the combined challenge of semantic alignment and R2RML mapping generation when attempted jointly in a one-shot setting. In comparison, traditional baselines such as Milan and A4MO demonstrate better performance (Table 2) but remain constrained by dependence on lexical similarity and predefined mapping templates.

Prior LLM-driven systems do not outperform the strongest traditional baseline. Table 2 shows that LLM4VKG (0.21), Table-to-KG (0.18), and Magneto (0.23) remain below Milan (0.31). This pattern indicates that simply introducing an LLM is not enough, performance depends on how the LLM is embedded in the pipeline.

The observed gap with Milan stems from limitations in earlier LLM-based approaches. Without decomposing the mapping task into sequential symbolic sub-stages that progressively constrain candidate sets using rule-based reasoning, the model faces a large, weakly constrained search space, making it more prone to imprecise or incorrect mappings. In addition, the lack of stage

Scenario	Conf	Mondial	NPD	Avg
Traditional Baselines				
AM4O [5]	0.41	0.44	0.23	0.26
Milan [13]	0.46	-	0.30	0.31
LLM Driven Baselines				
LLM4VKG [21]	0.51	0.18	0.19	0.21
Table-to-KG [20]#	0.33	0.10	0.17	0.18
Magneto [23]#	0.41	0.28	0.21	0.23
LLM Driven Demonstration Based Baselines				
Matchmaker [11, 12]	0.56	0.38	0.19	0.24
Manually Curated FS	0.72	0.82	0.48	0.53
Our Approach	0.79	0.79	0.51	0.56
Ablations				
- <i>Self-Demonstrations (FS)</i>	0.54	0.46	0.36	0.38
- <i>Matching</i>	0.51	0.50	0.36	0.38
- <i>Mapping</i> #	0.62	0.36	0.43	0.44
- <i>Matching+Mapping</i> †	0.46	0.08	0.07	0.10

Table 2.: Results on RODI Benchmark with Gemini-2.0-Flash. Conf: Conference-no-fk; NPD: Npd-Atomic; FS: Few-Shot; #: With programmatic mapping; †: No Decomposition.

specific supervision in the form of high quality few shot exemplars provides insufficient guidance on the expected behavior of different mapping sub-types. Element wise matching without sufficient contextualization further degrades performance; for example, when the model attempts to align a property of a given class without knowledge of other properties of that class, the table matched to the class, or relevant candidate columns along with representative example values, it may select a plausible but incorrect candidate due to the absence of sufficient context. Finally, reliance on a template based or programmatic mapping layer as in Milan limits expressiveness and cannot support multiple joins, unions, filtering conditions, or normalization rules required for complex mapping scenarios. Unlike prior LLM-driven approaches we exploit the complementary benefits of symbolic search space reduction and LLM-guided (beyond syntactic or semantic) candidate matching and SQL view generation to achieve substantial improvements.

RQ2: How does our symbolic decomposition-based LLM approach compare with traditional approaches? Our neuro-symbolic pipeline consistently outperforms traditional systems across all scenarios. In Table 2, our method achieves the best results on Conference-no-fk (0.79), Mondial (0.79), and NPD-Atomic (0.51), and substantially exceeds Milan, A4MO in average performance. The key reason is the complementary design of symbolic structure and neural reasoning. The symbolic layer decomposes the task into two stages: (i) matching (class, subclass, data property, object property) and (ii) mapping (SQL view generation with subject/object selection). This decomposition reduces task complexity and focuses each LLM call on a narrower, better-contextualized decision. The neural layer (LLM) complements this symbolic structure by going beyond name similarity to provide semantic and domain-specific disambiguation for cryptic schema elements using ontology context, schema structure, and representative column values, while also generating expressive SQL for non-trivial mappings.

Ablation results in Table 2 highlight the complementary roles of the matching and mapping stages. Dropping the matching stage substantially reduces performance, while replacing LLM-based mapping stage with programmatic mapping also results in a drop. These degradation indicate that both stages contribute essential, distinct capabilities to the pipeline, and their integration is critical for achieving high mapping quality.

RQ3: How do prior few-shot-based LLM approaches perform compared with prior approaches? Few-shot supervision is beneficial, but its effectiveness depends on the quality and breadth of the demonstrations. As shown in Table 2, manually well-curated few-shots substantially improve performance over traditional and LLM-driven baselines, underscoring the importance of in-context learning for this task. However, manually curating high-quality demonstrations is challenging and time-consuming at enterprise scale, requiring substantial domain expertise. As the number of ontology concepts and schema elements grows, identifying representative, diverse, and non-redundant few-shots becomes harder, making manual curation difficult to scale.

We evaluate a prior automatic few-shot selection strategy proposed in Matchmaker, originally developed for schema-to-schema matching. When adapted to our pipeline, Matchmaker-style automatic few-shot selection yields limited, inconsistent improvements and continues to perform below the strongest traditional baseline, Milan. A key limitation of automatically selected few-shots is their lack of coverage across the full range of mapping challenges. The selected examples tend to focus on straightforward semantic similarity cases, while failing to capture harder yet more consequential mapping patterns. In addition, Matchmaker’s selection strategy does not incorporate dependency-

aware context across stages, further degrading the coherence and correctness of the demonstrations. As a result, the selected few-shots fail to convey the behaviors needed for difficult cases, leading to reduced robustness and lower overall mapping quality.

RQ4: How does our dependency-aware, pattern-based self-demonstration approach compare with prior approaches? Our self-demonstration strategy addresses the above limitations by synthesizing stage-specific exemplars from reusable, domain-agnostic pattern families while respecting inter-stage dependencies. This produces demonstrations that are both diverse and structurally consistent with the pipeline’s execution order. As shown in Table 2, removing self-demonstrations lowers average performance to 0.38, underscoring the importance of high-quality supervision for the final outcome. An important and somewhat surprising observation is that, while the model performs poorly in zero-shot settings, enabling it to condition on self-generated few-shots guided by patterns and prior stage outputs leads to substantial performance gains. With self-demonstrations enabled, our full approach achieves an average score of 0.56, establishing state-of-the-art performance on Conference-no-fk, Mondial, and NPD-atomic.

Notably, our automatic self-demonstration strategy matches or slightly exceeds the performance of manually curated few-shots while fully eliminating manual curation overhead. The improvement is not due to simply increasing the number of examples, but due to generating the right demonstrations that are diverse, grounded in defined pattern families, and aligned with inter-stage dependencies in the pipeline.

RQ5: Is our approach generalizable across LLMs? We perform a comparative analysis using GPT-4o to assess the influence of LLM choice. The LLM-driven baselines are compared with our approach on the Conference-no-FK and Mondial scenarios. The NPD scenario, being significantly larger, is omitted from this specific evaluation due to budget limitations. Table 3 presents these results, indicating that performance with GPT-4o is lower than Gemini-2.0-Flash (Table 2). With GPT-4o, our approach consistently surpasses Table-to-KG and Magneto across scenarios, demonstrating its generalizability across LLMs.

Overall, the answers to RQ1–RQ5 converge on a clear conclusion. Strong performance on DB-to-ontology mapping does not emerge from LLM capability alone, it depends on how the task is organized and supervised. In particular, symbolic decomposition improves tractability and dependency-aware pattern-guided self-demonstrations improve decision quality at each stage, and expressive mapping generation is essential for resolving structurally complex correspondences.

Error Analysis: We analyze errors for the most complex NPD-Atomic scenario. Its 439 test query pairs evaluate mapping outcomes for distinct ontology concepts: 134 classes, 213 data properties, and 92 object properties. Our approach yields incorrect mappings for 46 classes, 157 data properties, and 63 object properties. We randomly sample and analyze 20 test queries for each ontology concept.

For classes, 12 errors stem from incorrect matches, 6 from incorrect SQL views, and 2 from incorrect subject/object column selection. Incorrect matches mainly arise when a class corresponds to multiple tables or when predicted and actual tables are ambiguous. For example, the `Jacket4LegsFacility` class matches both `facility_fixed` and

Scenario	Conf	Mondial	Avg
Table-to-KG [20]#	0.21	0.13	0.17
Magneto [23]#	0.49	0.34	0.40
Our approach	0.56	0.65	0.61

Table 3.: Results on RODI Benchmark with GPT4o. Conf: Conference-no-FK; #: With programmatic mapping.

May 2026

facility_moveable tables (where fclkind = 'JACKET 4 LEGS'), but our approach identifies only facility_fixed, relying on the definition of 'Jacket' as a fixed offshore structure anchored to the seabed. Similarly, ParcelBAA class is incorrectly mapped to baaarea table instead of bsns_arr_area table, because both tables share similar column names and contain baakind column with value 'PARCELL', making them hard to distinguish. SQL-view errors involve (i) incorrect or missing categorical filter values and (ii) incorrect column selection. For example, although Onshorefacility class is correctly matched to column fclkind, the generated SQL filters by 'LANDFALL' instead of 'ONSHORE FACILITY'. Likewise, for LithostratigraphicUnit class, correctly matched to wellbore_formation_top table, the SQL view selects lsuname and lsulevel columns instead of all primary keys of the table.

For data properties, 10 errors are due to incorrect matches and 10 to incorrect SQL view generation. Most incorrect-match errors stem from incorrect domain class inferences during the prior metadata inference step. For example, the data property sourcePressure is inferred to belong to SeismicSurvey class. This inference is semantically plausible, since source pressure relates to seismic acquisition, but the ground-truth mapping assigns it to SurveyArea and SurveyMultilineArea. These more specific domain classes are difficult to infer from the property name alone. SQL-generation errors involve incorrect column selection or missing information. For example, although dateStatusTo data property is correctly matched to the field_activity_status_hst column, the generated SQL view does not filter out the placeholder date '9999-12-31T00:00:00', leading to errors. A particularly challenging case is coreIntervalTop property of WellboreCore class, correctly matched to wlbCoreIntervalTop column in wellbore_core table. The ground-truth SQL view is as follows:

```
(SELECT wellbore_core_id, wlbnpdidwellbore,
wlbcorenumber, wlbcoreintervaltop FROM
wellbore_core WHERE wlbcoreintervaluom = '[m]')
UNION ALL (SELECT wellbore_core_id,
wlbnpdidwellbore, wlbcorenumber,
wlbcoreintervaltop * 0.3048 FROM wellbore_core
WHERE wlbcoreintervaluom = '[ft]')
```

The ontology does not specify that all values must be normalized to meters using wlbCoreIntervalUom column, making the correct SQL view impossible for the LLM to generate.

For object properties, 16 errors are due to incorrect matches and 4 to incorrect SQL views. Among incorrect-match errors, 6 stem from wrongly inferred domain/range classes and 3 from incorrect domain/range matches. SQL-view errors usually involve incorrect column selection or missing information. For example, the LLM fails to correctly map the object property productionForField because the reason for excluding value '44576' is not specified. The ground-truth SQL view is:

```
SELECT prfnpdidininformationcarrier, prfyear
FROM field_production_yearly
WHERE prfnpdidininformationcarrier <> '44576'.
```

6. Conclusion

Enterprise-scale schema-to-ontology mapping remains a challenging problem due to severe semantic heterogeneity, noisy and incomplete metadata, cryptically named schema elements, and the fundamental mismatch in abstraction between relational data and ontological models. This work shows that while LLMs offer strong semantic reasoning capabilities, they are ineffective when applied in a naïve or weakly structured manner. Our study establishes that a neuro-symbolic approach is essential for achieving reliable and accurate mappings. By decomposing the task into structured stages and using symbolic constraints to systematically narrow the search space, the approach improves tractability and enables LLMs to operate within focused contexts leading to improved accuracy.

At the same time, decomposition alone is insufficient to fully address the challenges of schema-to-ontology mapping. An important finding of this work is the surprising effectiveness of self-demonstrations in supervising each stage of the pipeline. Self-demonstrations refer to automatically generated in-context examples produced by the language model itself, showing how specific matching or mapping decisions should be made for a given sub-task. These examples are guided by reusable, domain-agnostic mapping patterns, ensuring coverage of common and complex correspondence behaviors observed in real integration settings. By tailoring self-demonstrations to individual sub-tasks, the model receives explicit and diverse guidance that clarifies expected reasoning and outputs. Furthermore, generating these examples in a dependency-aware manner, where downstream stages are conditioned on the outputs of earlier ones, preserves consistency across stages and limits error propagation. Together, these properties enable more reliable local decisions, which accumulate to produce more accurate mappings. Leveraging the neuro-symbolic decomposition together with pattern-guided context-aware self-demonstrations, our method establishes state-of-the-art performance on the three most challenging scenarios from the RODI benchmark, demonstrating ~ 25 percentage points improvements in the F1 score over prior automated baselines.

As future work, we plan to reduce cost by partially replacing LLM-based reasoning with programmatic and template-driven mapping for well-structured and repetitive correspondence patterns. In addition, instead of producing a single mapping decision, we will output ranked candidate matches accompanied by confidence scores, enabling expert-in-the-loop workflows where domain experts can efficiently validate, adjust, or correct mappings by inspecting a small set of plausible alternatives rather than performing exhaustive manual verification.

Declaration on Generative AI

During the preparation of this work, the author(s) used Microsoft Copilot in order to paraphrase and reword portions of human-authored text. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication’s content.

Tools and services: Microsoft Copilot.

Tools’ contributions: Paraphrasing and rewording (linguistic refinement) of human-authored text.

References

- [1] Buron M, Goasdoué F, Manolescu I, Mugnier ML. Obi-Wan: ontology-based RDF integration of heterogeneous data. *Proc VLDB Endow.* 2020 Aug;13(12):2933–2936. Available from: <https://doi.org/10.14778/3415478.3415512>.
- [2] Ohmori Y, Kitagawa H, Amagasa T, Matono A. Integration of knowledge bases and external sources using user-defined predicates and adaptive entity linking. *World Wide Web.* 2025 Jun;28(4). Available from: <https://doi.org/10.1007/s11280-025-01355-x>.
- [3] Rodríguez-Muro M, Kontchakov R, Zakharyashev M. Ontology-based data access: Ontop of databases. In: *International Semantic Web Conference*. Springer; 2013. p. 558-73.
- [4] Pinkel C, Binnig C, Jiménez-Ruiz E, Kharlamov E, May W, Nikolov A, et al. RODI: Benchmarking relational-to-ontology mapping generation quality. *Semantic Web.* 2017;9:25-52. Available from: <https://api.semanticscholar.org/CorpusID:21304033>.
- [5] Sicilia Á, Nemirovski G. AutoMap4OBDA: Automated Generation of R2RML Mappings for OBDA. In: *International Conference Knowledge Engineering and Knowledge Management*; 2016. Available from: <https://api.semanticscholar.org/CorpusID:38363184>.
- [6] Kojima T, Gu SS, Reid M, Matsuo Y, Iwasawa Y. Large Language Models are Zero-Shot Reasoners. *ArXiv.* 2022;abs/2205.11916. Available from: <https://api.semanticscholar.org/CorpusID:249017743>.
- [7] Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, et al. Language Models are Few-Shot Learners. *ArXiv.* 2020;abs/2005.14165. Available from: <https://api.semanticscholar.org/CorpusID:218971783>.
- [8] Wei J, Tay Y, Bommasani R, Raffel C, Zoph B, Borgeaud S, et al. Emergent Abilities of Large Language Models. *ArXiv.* 2022;abs/2206.07682. Available from: <https://api.semanticscholar.org/CorpusID:249674500>.
- [9] Wei J, Wang X, Schuurmans D, Bosma M, Chi EH, Xia F, et al. Chain of Thought Prompting Elicits Reasoning in Large Language Models. *ArXiv.* 2022;abs/2201.11903. Available from: <https://api.semanticscholar.org/CorpusID:246411621>.
- [10] Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, et al. Language Models are Few-Shot Learners. In: Larochelle H, Ranzato M, Hadsell R, Balcan MF, Lin H, editors. *Advances in Neural Information Processing Systems*. vol. 33. Curran Associates, Inc.; 2020. p. 1877-901.
- [11] Seedat N, van der Schaar M. Matchmaker: Self-Improving Large Language Model Programs for Schema Matching. *ArXiv.* 2024;abs/2410.24105. Available from: <https://api.semanticscholar.org/CorpusID:273707861>.
- [12] Seedat N, van der Schaar M. Bootstrapping Self-Improvement of Language Model Programs for Zero-Shot Schema Matching. In: *Forty-second International Conference on Machine Learning*; 2025. .
- [13] Mathur S, O’Sullivan D, Brennan R. Milan: Automatic Generation of R2RML Mappings. In: *Irish Conference on Artificial Intelligence and Cognitive Science*; 2018. Available from: <https://api.semanticscholar.org/CorpusID:49570848>.
- [14] Jiménez-Ruiz E, Kharlamov E, Zheleznyakov D, Horrocks I, Pinkel C, Skjæveland MG, et al. BootOX: Practical Mapping of RDBs to OWL 2. In: *International Workshop on the Semantic Web*; 2015. Available from: <https://api.semanticscholar.org/CorpusID:2287280>.
- [15] Calvanese D, Cogrel B, Komla-Ebri S, Kontchakov R, Lanti D, Rezk M, et al. Ontop: Answering SPARQL queries over relational databases. *Semantic Web.* 2016;8:471-87. Available from: <https://api.semanticscholar.org/CorpusID:7721653>.
- [16] de Medeiros LF, Priyatna F, Corcho Ó. MIRROR: Automatic R2RML Mapping Generation from Relational Databases. In: *International Conference on Web Engineering*; 2015. Available from: <https://api.semanticscholar.org/CorpusID:27007091>.
- [17] Oldakowski R. D2RQ Platform – Treating Non-RDF Databases as Virtual RDF Graphs. *Nature Precedings.* 2011. Available from: <https://api.semanticscholar.org/CorpusID:263364806>.
- [18] Aumüller D, Do HH, Maßmann S, Rahm E. Schema and ontology matching with COMA++. In: *ACM SIGMOD Conference*; 2005. Available from: <https://api.semanticscholar.org/CorpusID:13169021>.
- [19] Pinkel C, Binnig C, Jiménez-Ruiz E, Kharlamov E, Nikolov A, Schwarte A, et al. IncMap: A Journey towards Ontology-based Data Integration. In: *Datenbanksysteme für Business, Technologie und Web*; 2017. Available from: <https://api.semanticscholar.org/CorpusID:38803389>.

- [20] Vandemoortele N, Steenwinckel B, Hoecke SV, Ongenae F. Scalable Table-to-Knowledge Graph Matching from Metadata using LLMs. In: SemTab@ISWC; 2024. Available from: <https://api.semanticscholar.org/CorpusID:275571188>.
- [21] Xiao G, Ren L, Qi G, Xue H, Panfilo MD, Lanti D. LLM4VKG: Leveraging Large Language Models for Virtual Knowledge Graph Construction. In: International Joint Conference on Artificial Intelligence; 2025. Available from: <https://api.semanticscholar.org/CorpusID:281477725>.
- [22] Sheetrit E, Brief M, Mishaeli M, Elisha O. ReMatch: Retrieval Enhanced Schema Matching with LLMs. ArXiv. 2024;abs/2403.01567. Available from: <https://api.semanticscholar.org/CorpusID:268248876>.
- [23] Liu Y, Pena EHM, Santos ASR, Wu E, Freire J. Magneto: Combining Small and Large Language Models for Schema Matching. ArXiv. 2024;abs/2412.08194. Available from: <https://api.semanticscholar.org/CorpusID:274638317>.
- [24] Das S, Sundara S, Cyganiak R. R2RML: RDB to RDF Mapping Language. World Wide Web Consortium (W3C); 2012. REC-r2rml-20120927. Available from: <https://www.w3.org/TR/r2rml/>.
- [25] Du Y, Tian M, Ronanki S, Rongali S, Bodapati S, Galstyan AG, et al. Context Length Alone Hurts LLM Performance Despite Perfect Retrieval; 2025. Available from: <https://api.semanticscholar.org/CorpusID:281826429>.
- [26] Hsieh CP, Sun S, Krizan S, Acharya S, Rekesh D, Jia F, et al. RULER: What's the Real Context Size of Your Long-Context Language Models? ArXiv. 2024;abs/2404.06654. Available from: <https://api.semanticscholar.org/CorpusID:269032933>.
- [27] Shi F, Chen X, Misra K, Scales N, Dohan D, Chi EH, et al. Large Language Models Can Be Easily Distracted by Irrelevant Context. In: International Conference on Machine Learning; 2023. Available from: <https://api.semanticscholar.org/CorpusID:256459776>.
- [28] Yang M, Huang E, Zhang L, Surdeanu M, Wang WY, Pan L. How Is LLM Reasoning Distracted by Irrelevant Context? An Analysis Using a Controlled Benchmark. ArXiv. 2025;abs/2505.18761.
- [29] Kapuriya J, Kaushik M, Ganguly D, Bhatia SK. Exploring the Role of Diversity in Example Selection for In-Context Learning. Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval. 2025. Available from: <https://api.semanticscholar.org/CorpusID:278327428>.
- [30] Thorpe DG, Duberstein A, Kinsey IA. Dubo-SQL: Diverse Retrieval-Augmented Generation and Fine Tuning for Text-to-SQL. ArXiv. 2024;abs/2404.12560. Available from: <https://api.semanticscholar.org/CorpusID:269282528>.
- [31] Wang X, Wei J, Schuurmans D, Le Q, Chi EH, Zhou D. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ArXiv. 2022;abs/2203.11171. Available from: <https://api.semanticscholar.org/CorpusID:247595263>.
- [32] Song K, Tan X, Qin T, Lu J, Liu TY. MPNet: Masked and Permuted Pre-training for Language Understanding. ArXiv. 2020;abs/2004.09297. Available from: <https://api.semanticscholar.org/CorpusID:215827489>.